

Rate Limiting

14 August 2026 04:41 PM

What is Rate Limiting?

Rate limiting controls how many requests a client can make within a certain amount of time.

Example:

Limit = 5 requests / second

Request 1 → ☒

Request 2 → ☒

Request 3 → ☒

Request 4 → ☒

Request 5 → ☒

Request 6 → ☒

Why do we need it?

Rate limiting protects our system from:

- Too many requests
- Abuse
- DDoS-like traffic
- One user consuming all resources
- Sudden traffic spikes

There are **5 common algorithms**:

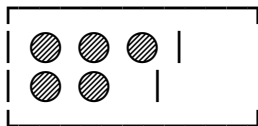
1. Token Bucket
2. Leaky Bucket
3. Fixed Window Counter
4. Sliding Window Log
5. Sliding Window Counter

1. Token Bucket

Idea

Imagine a bucket containing tokens.

Bucket capacity = 5 tokens



Tokens are added at a fixed rate.

For example:

10 tokens / second

Every request needs **1 token**.

Request

↓

Token available?

↓

Yes → Remove token → Allow

No → Reject

Example

Capacity = 5

Refill = 1 token/second

Initially:

5 tokens

Five requests can happen immediately:

Request 1 → 4 tokens

Request 2 → 3 tokens

Request 3 → 2 tokens

Request 4 → 1 token

Request 5 → 0 tokens

Request 6 → ✗

After 1 second:

1 new token

So another request can pass.

C# Implementation

```
public class TokenBucket
{
    private readonly int capacity;
    private readonly double refillRate;
    private double tokens;
    private DateTime lastRefill;
    public TokenBucket(int capacity, double refillRate)
    {
        this.capacity = capacity;
        this.refillRate = refillRate;
        tokens = capacity;
        lastRefill = DateTime.UtcNow;
    }
    public bool AllowRequest()
    {
        RefillTokens();
        if (tokens >= 1)
        {
            tokens--;
            return true;
        }
        return false;
    }
    private void RefillTokens()
    {
        var now = DateTime.UtcNow;
        var elapsedSeconds =
            (now - lastRefill).TotalSeconds;
        tokens = Math.Min(
            capacity,
            tokens + elapsedSeconds * refillRate
        );
        lastRefill = now;
    }
}
```


2. Leaky Bucket

Idea

Imagine a bucket with a hole.

Requests enter the bucket:

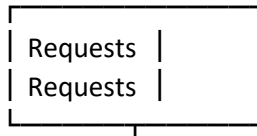
Request

Request

Request

Request

↓



↓

1 request

per second

The bucket processes requests at a **constant rate**.

So if 10 requests arrive together:

10 requests

↓

Queue

↓

1 → 1 → 1 → 1 → ...

This **smooths traffic**.

C# Simple Implementation

```
using System.Collections.Concurrent;
public class LeakyBucket
{
    private readonly int capacity;
    private readonly TimeSpan interval;
    private readonly Queue<DateTime> queue = new();
    private DateTime lastProcessed = DateTime.MinValue;
    public LeakyBucket(int capacity, TimeSpan interval)
    {
        this.capacity = capacity;
        this.interval = interval;
    }
    public bool AllowRequest()
    {
        lock (queue)
        {
            if (queue.Count >= capacity)
            {
                return false;
            }
            queue.Enqueue(DateTime.UtcNow);
            return true;
        }
    }
    public void Process()
```



```

{
lock (queue)
{
if (queue.Count == 0)
return;
if (DateTime.UtcNow - lastProcessed >= interval)
{
queue.Dequeue();
lastProcessed = DateTime.UtcNow;
Console.WriteLine("Request processed");
}
}
}
}

```

Difference from Token Bucket

Token Bucket:

Requests → Tokens → Process
Can allow a burst.

Leaky Bucket:

Requests → Queue → Process at constant rate
Smooths the traffic.

Remember

Leaky Bucket = Smooth the traffic.

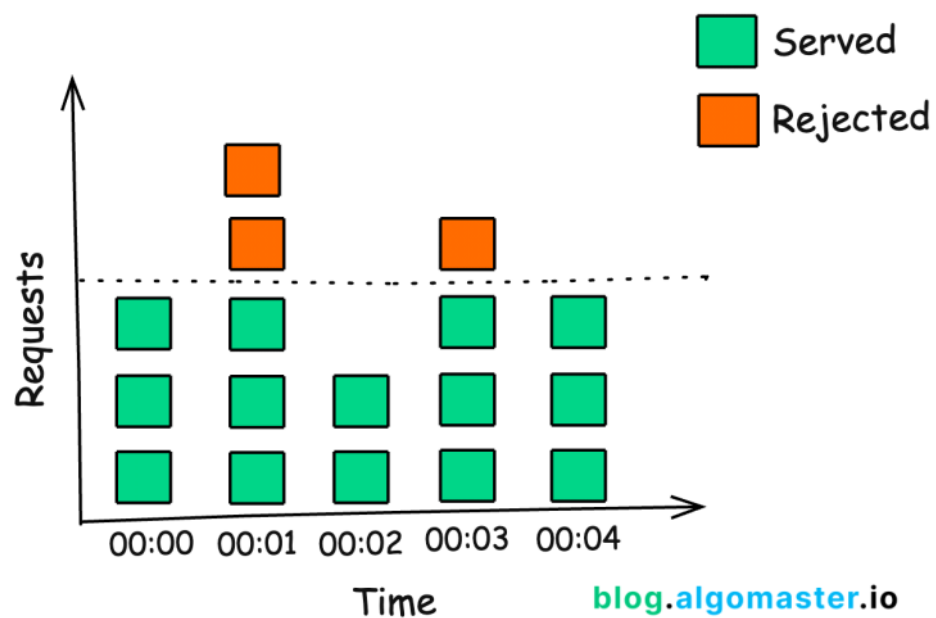
Pros:

- Consistent processing rate
- Prevents sudden bursts

Cons:

- Bursts can be dropped when bucket is full
- Slightly more complex

3. Fixed Window Counter





Divide time into fixed windows.

Example:

Limit = 5 requests / minute

Time:

10:00:00 ————— 10:01:00

Window 1

10:01:00 ————— 10:02:00

Window 2

Each window has a counter.

Window 1

Counter = 0

Request → 1

Request → 2

Request → 3

Request → 4

Request → 5

Request → ✕

At 10:01:00, counter resets:

Counter = 0

C# Implementation

```
public class FixedWindowLimiter
{
    private readonly int limit;
    private readonly TimeSpan window;
    private int count;
    private DateTime windowStart;
    public FixedWindowLimiter(int limit, TimeSpan window)
    {
        this.limit = limit;
        this.window = window;
        windowStart = DateTime.UtcNow;
    }
    public bool AllowRequest()
    {
        {
            var now = DateTime.UtcNow;
            if (now - windowStart >= window)
            {
                count = 0;
                windowStart = now;
            }
            if (count >= limit)
            {
                return false;
            }
            count++;
            return true;
        }
    }
}
```


Problem

The biggest problem is the **window boundary**.

Suppose:

Limit = 5 requests/minute

User sends:

10:00:59 → 5 requests

10:01:01 → 5 requests

That's **10 requests in about 2 seconds**.

So Fixed Window can allow bursts around boundaries.

Remember

Fixed Window = Simple counter that resets after every fixed period.

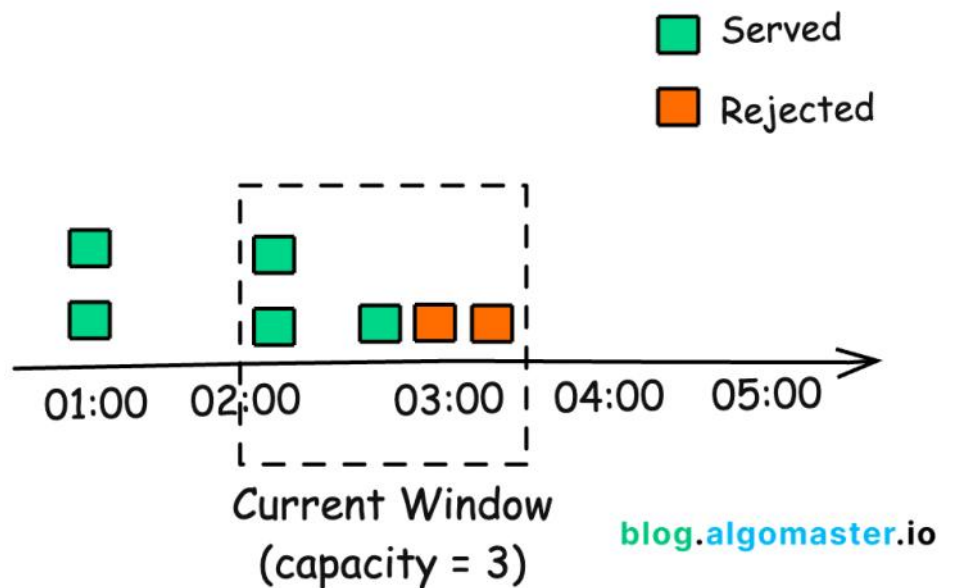
Pros:

- Very simple
- Easy to understand

Cons:

- Boundary burst problem

4. Sliding Window Log



Idea

Instead of counting one fixed window, keep the **timestamp of every request**.

Example:

Limit = 5 requests / 60 seconds

Store:

10:00:10

10:00:20

10:00:25

10:00:40

10:00:50

New request arrives at:

10:00:55

Count timestamps from the last 60 seconds.
If:

count < 5
allow the request.
Otherwise:

X Reject

C# Implementation

```
public class SlidingWindowLog
{
    private readonly int limit;
    private readonly TimeSpan window;
    private readonly Queue<DateTime> requests = new();
    public SlidingWindowLog(int limit, TimeSpan window)
    {
        this.limit = limit;
        this.window = window;
    }
    public bool AllowRequest()
    {
        var now = DateTime.UtcNow;
        while (
            requests.Count > 0 &&
            now - requests.Peek() >= window)
        {
            requests.Dequeue();
        }
        if (requests.Count >= limit)
        {
            return false;
        }
        requests.Enqueue(now);
        return true;
    }
}
```

Why is it accurate?

Because it looks at the **actual timestamps** of requests.
There is no fixed-window boundary problem.

Problem

Suppose you have:

1 million users
100 requests/user
You might need to store a huge number of timestamps.
So memory usage can become large.

Remember

Sliding Window Log = Store every request timestamp.

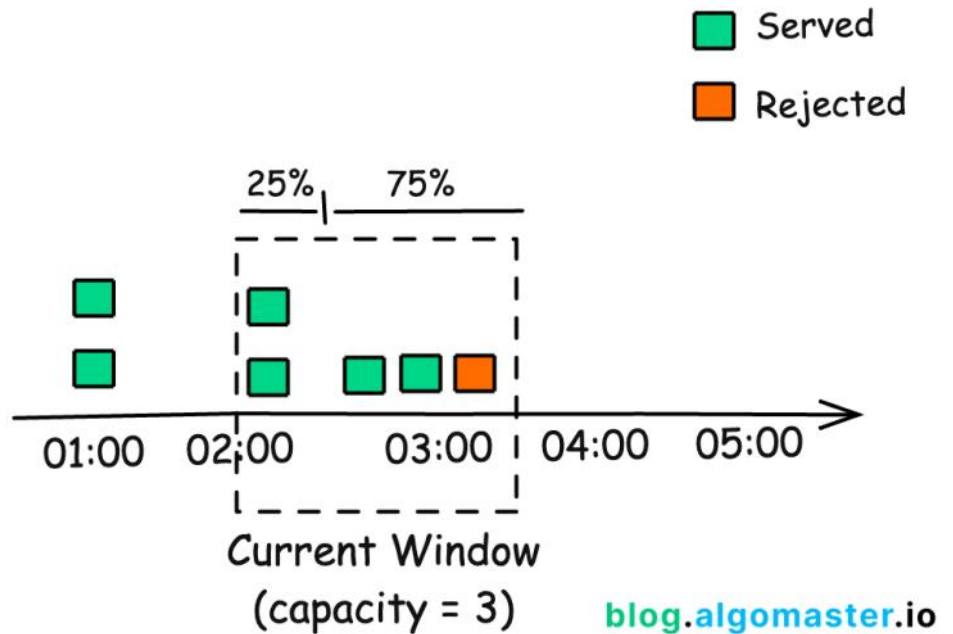
Pros:

- Very accurate
- No window-boundary issue

Cons:

- Uses more memory
- Expensive for high traffic

5. Sliding Window Counter



This combines ideas from:

Fixed Window Counter

+

Sliding Window

Instead of storing every timestamp, we store **request counts**.

Suppose:

Limit = 100 requests/minute

We keep:

Previous window = 80 requests

Current window = 30 requests

If we're 75% into the current window:

25% of previous window

+

100% of current window

Estimated requests:

$$80 \times 25\% + 30$$

$$= 20 + 30$$

$$= 50$$

Then we can decide whether the next request should be allowed.

C# Simple Implementation

```
public class SlidingWindowCounter
{
    private readonly int limit;
    private readonly TimeSpan window;
    private DateTime currentWindowStart;
    private int currentCount;
```



```

private int previousCount;
public SlidingWindowCounter(
    int limit,
    TimeSpan window)
{
    this.limit = limit;
    this.window = window;
    currentWindowStart = DateTime.UtcNow;
}
public bool AllowRequest()
{
    var now = DateTime.UtcNow;
    var elapsed =
        now - currentWindowStart;
    if (elapsed >= window)
    {
        previousCount = currentCount;
        currentCount = 0;
        currentWindowStart = now;
        elapsed = TimeSpan.Zero;
    }
    double currentWindowProgress =
        elapsed.TotalMilliseconds /
        window.TotalMilliseconds;
    double previousWeight =
        1 - currentWindowProgress;
    double estimatedCount =
        previousCount * previousWeight +
        currentCount;
    if (estimatedCount >= limit)
    {
        return false;
    }
    currentCount++;
    return true;
}
}

```

Why use it?

You don't need to store every request timestamp.
Instead:

Previous count

Current count

So it uses less memory than Sliding Window Log.

Remember

Sliding Window Counter = More accurate than Fixed Window, cheaper than Sliding Window Log.

Pros:

- More accurate
- Lower memory usage
- Smooths window boundaries

Cons:

- More complicated
- Uses an approximation

